

Process Orchestration

Networks blip. APIs throttle. Humans misclick. Production AI lives or dies by its orchestration — how cleanly it retries, branches, and observes.

Name one pipeline of yours that breaks when re-run. That is our target today.

WHAT'S INSIDE

- 01** Choreography vs orchestration
- 02** The four verbs
- 03** Idempotence — the non-negotiable
- 04** Sagas — compensating actions
- 05** Example IDP workflow

WHY THIS LESSON MATTERS

The pipeline will fail. Engineer for that.

WHY

Real

A real reason this matters in industry today.

SCALE

Today

A real reason this matters in industry today.

SPEED

Now

A real reason this matters in industry today.

IMPACT

Yours

A real reason this matters in industry today.

LEARNING OUTCOMES

By the end of this lesson, you will be able to —

1

Choreography vs orchestration.

2

Design multi-step workflows with decision gates + retries.

3

Airflow / Prefect / Temporal at a conceptual level.

4

Spot idempotence failures that break pipelines.

5

Implement a 3-step orchestration in Python.

AGENDA

What we'll cover today

Short, sequenced parts. Each one builds on the last.

01

Choreography vs...

Most AI pipelines are better orchestrated than choreographed.

02

The four verbs

Sequence · Retry · Gate · Observe

03

Idempotence — the...

Every step must be safely re-runnable

04

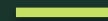
Sagas — compensating actions

Undo when the chain fails

01

PART 1 OF 4

Choreography vs...



Most AI pipelines are better orchestrated than choreographed.

CONCEPT 1 OF 4

Choreography vs orchestration

Most AI pipelines are better orchestrated than choreographed.

DEFINITION

Choreography vs orchestration

Most AI pipelines are better orchestrated than choreographed.

EXAMPLE

Real-world example: applies directly to Indian industry contexts.

CONCEPT 2 OF 4

The four verbs

SEQUENCE — define step order.

RETRY — backoff on transient failure.

GATE — branch on step output (decision gates).

OBSERVE — log everything; measure step timing.

MEMORISE

S · R · G · O

Any orchestrator must do all four.

KEY POINTS

- SEQUENCE — define step order
- RETRY — backoff on transient failure
- GATE — branch on step output (decision gates)
- OBSERVE — log everything; measure step timing

CONCEPT 3 OF 4

Idempotence — the non-negotiable

Key every write by a business ID (invoice_id, order_id).
Check "already processed" before performing the side effect.
Retries + at-least-once delivery will happen.
Not idempotent → duplicate side effects → data corruption.

RULE

Safely re-runnable

If a step cannot be retried, it is not ready for production.

DEFINITION

Idempotence — the non-negotiable

Every step must be safely re-runnable

EXAMPLE

Real-world example: applies directly to Indian industry contexts.

Sagas — compensating actions

Each step has a "compensate" (undo).

If step N fails, run `compensate(N-1)`, `compensate(N-2)`, ...

Critical in money-moving pipelines.

Design compensations when you design the happy path.

KEY POINTS

- Each step has a "compensate" (undo)
- If step N fails, run `compensate(N-1)`, `compensate(N-2)`, ...
- Critical in money-moving pipelines
- Design compensations when you design the happy path

02

PART 2 OF 4

Engage and reflect

Pause, talk, and connect what you just learned to your own work.

✦ THINK • PAIR •
SHARE

ACTIVITY

Orchestrate Flipkart return-refund

PROMPT

Apply the four concepts above to one real example from your own week.

HOW TO RUN IT

1

2

3

4

Reveal: instructor confirms the canonical answer.

REFLECT & APPLY

Three questions before you close this lesson

Spend 60 seconds on each. Write the answers down — no scrolling, no shortcuts.

Q1

Which pipeline of yours would break on retry today?

Q2

Where would you add a saga?

Q3

Which tool — Airflow, Prefect, or Temporal — fits your current team?

03

PART 3 OF 4

Knowledge check

Three short questions. One minute each. Honest answers only.

Central coordinator = ?

A

Choreography

B

Orchestration

C

Event bus

D

Microservice

Central coordinator = ?



CORRECT ANSWER

Orchestration

WHY

Orchestration has a single coordinator; choreography is event-driven without one

Every step **MUST** be:

A

Async

B

Idempotent

C

In Python

D

Stateful

Every step MUST be:



CORRECT ANSWER

Idempotent

WHY

Retries + at-least-once delivery demand idempotence

Saga purpose:



Faster execution



Compensating undo



More steps



Encryption

Saga purpose:



CORRECT ANSWER

Compensating undo

WHY

Saga = per-step compensating actions to undo partial failure

04

PART 4 OF 4

Apply and close

One thing to remember. One thing to ship this week.

SUMMARY

Five sentences to remember

If you remember nothing else from this lesson, remember these five lines.

- 1 Pick the right tool — never the loudest one.
- 2 Practice the framework on a real example.
- 3 Watch for the three classic pitfalls.
- 4 Document your decision in one sentence.
- 5 Carry the discipline forward to the next lesson.

WORKED EXAMPLE

How this lesson plays out in one real Indian context

THE SCENARIO

Most AI pipelines are better orchestrated than choreographed.

THE TAKEAWAY

"Four verbs of every orchestration. Miss one; production bites."

WHAT TO NOTICE

1

Choreography vs...

2

The four verbs

3

Idempotence — the...

4

Sagas — compensating actions

Mini assignment — orchestration diagram

Sketch an orchestration for a Flipkart return-refund (7+ steps, 2 gates, 1 human review).

DELIVERABLES

- Draw as a DAG.
- Mark idempotent steps explicitly.
- Show the compensating actions for a saga.
- Post diagram on the forum.

WHAT 'GOOD' LOOKS LIKE

Deliverable: PNG diagram on the cohort forum.

“

"Four verbs of every orchestration. Miss one; production bites."

— AI Learning Hub

END OF LESSON 4

What you can now do

Each one of these is now part of your working vocabulary.

- 1 Pick the right tool — never the loudest one.
- 2 Practice the framework on a real example.
- 3 Watch for the three classic pitfalls.
- 4 Document your decision in one sentence.
- 5 Carry the discipline forward to the next lesson.

NEXT → Lesson 7 — Connecting RPA bots to AI models.